
DevOps Preparation Guide

1. A wrong merge causes a production outage. How would you investigate?

Answer:

- Identify the merge commit using git log.
- Compare changes with the previous stable release using git diff.
- Review CI/CD logs.
- Verify who merged and whether code review was completed.
- Roll back using git revert or redeploy the last stable release.

2. Critical code is accidentally overwritten. What would you do first?

Answer:

- Stop further changes.
- Check git reflog or commit history.
- Restore the correct commit using git checkout, git restore, or git revert.
- Validate the restored code before redeployment.

3. A deployment uses the wrong Git branch. How would you troubleshoot?

Answer:

- Verify the deployed branch (git branch, CI/CD variables).
- Compare deployed commit with the intended branch.
- Review pipeline configuration and deployment logs.
- Redeploy from the correct branch.

4. Git history is rewritten accidentally. How would you recover?

Answer:

- Use git reflog to locate previous commits.
- Restore the lost commit or branch.
- If pushed remotely, coordinate with the team before force-pushing corrected history.

5. CI/CD pipelines fail after a repository change. What would you investigate?

Answer:

- Review recent commits.
- Check pipeline logs.
- Verify YAML/Jenkinsfile changes.
- Validate branch permissions, secrets, dependencies, and build scripts.
- Run the pipeline locally if possible.

6. A force push impacts multiple developers. How would you respond?

Answer:

- Pause further pushes.
- Recover commits using git reflog.
- Communicate with the team.
- Restore the branch from a known-good state.
- Restrict force pushes on protected branches.

7. Tags point to the wrong release version. How would you troubleshoot?

Answer:

- Verify the commit associated with the tag.
- Compare with the expected release commit.
- Delete and recreate the correct tag.
- Update the deployment if required.

8. Sensitive files are committed to the repository. How would you handle it?

Answer:

- Rotate exposed secrets immediately.
- Remove sensitive data from Git history using history-rewrite tools.
- Force-push only after team coordination.
- Add secrets to .gitignore and enable secret scanning.

9. Git conflicts delay a critical deployment. How would you resolve them?

Answer:

- Pull the latest changes.
- Resolve conflicts manually.
- Test the merged code.
- Commit the resolution.
- Perform peer review before deployment.

10. Production code differs from the repository. How would you investigate?

Answer:

- Compare deployed commit with Git.
- Verify deployment logs.
- Check for manual changes on the server.
- Confirm the correct artifact was deployed.
- Restore production from the repository if needed.

11. Rollback requires restoring a previous commit. How would you approach it?

Answer:

- Identify the last stable commit.
- Use git revert for shared branches.
- Redeploy the reverted version.
- Validate application health after rollback.

12. Branch protection rules are bypassed. What risks would you assess?

Answer:

- Unauthorized code changes
- Unreviewed commits
- Failed quality checks
- Security vulnerabilities
- Increased production outage risk

13. Repository access changes impact deployments. How would you troubleshoot?

Answer:

- Check repository permissions.

- Verify deploy keys or tokens.
- Review CI/CD credentials.
- Check audit logs.
- Validate branch access policies.

14. Monitoring shows healthy infrastructure but new code fails. What would you investigate?

Answer:

- Review application logs.
- Compare recent commits.
- Validate configuration changes.
- Check database connectivity.
- Roll back if the new release introduced the issue.

15. Multiple releases are merged together accidentally. How would you analyze the impact?

Answer:

- Identify merged commits.
- Review release notes.
- Determine affected features.
- Test impacted functionality.
- Roll back or revert only the problematic changes if possible.

16. A Git issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Git commit history
- Merge requests
- CI/CD logs
- Deployment logs
- Audit logs
- Application logs
- Timeline of events

- Root cause and recovery actions

17. Which Git commands are most useful during incidents?

Answer:

git status
git log
git diff
git show
git reflog
git blame
git branch
git checkout
git revert
git reset

18. What logs would you review during failures?

Answer:

- Git commit history
- Pull request/Merge request logs
- CI/CD logs
- Jenkins/GitHub Actions/Azure DevOps logs
- Deployment logs
- Application logs
- Kubernetes/Docker logs
- Audit logs

19. How would you validate code before merging?

Answer:

- Code review
- Unit tests
- Integration tests
- Static code analysis
- Security scanning
- CI pipeline success

- Merge approval

20. What controls reduce source-code risks?

Answer:

- Branch protection
- Pull request reviews
- CI/CD validation
- Secret scanning
- Signed commits
- Role-based access control (RBAC)
- Automated testing

21. How would you improve release governance?

Answer:

- Use release branches
- Require approvals
- Maintain release checklists
- Automate deployments
- Enable rollback plans
- Track releases with tags
- Maintain audit trails

22. What testing should occur before production releases?

Answer:

- Unit testing
- Integration testing
- Regression testing
- Security testing
- Performance testing
- Smoke testing
- User Acceptance Testing (UAT)

23. What should be documented in a Git RCA?

Answer:

- Incident summary
- Timeline
- Root cause
- Impact
- Affected commits
- Resolution
- Rollback details
- Lessons learned
- Preventive actions

24. How would you improve code visibility?

Answer:

- Pull requests
- Code reviews
- Branch strategy
- Commit standards
- Release notes
- Dashboards
- Notifications
- Repository audit logs

25. What preventive actions would reduce future incidents?

Answer:

- Enforce branch protection
- Mandatory code reviews
- Automated CI/CD validation
- Comprehensive testing
- Signed releases

- Feature flags
- Secret scanning
- Deployment approvals
- Rollback procedures
- Continuous monitoring

Q: What is your Git troubleshooting approach during a production incident?

Answer:

"I first identify the deployed commit and compare it with the expected release. I review recent merges, commit history, deployment and CI/CD logs, and verify whether the correct branch, tag, and artifact were used. If the issue is caused by a code change, I isolate the offending commit using git log, git diff, and git show, then either revert the change or roll back to the last stable release. After service restoration, I perform a root cause analysis and implement preventive measures such as branch protection, mandatory reviews, automated testing, and release approvals."

1. Containers cannot communicate with each other. How would you investigate?

Answer:

- Verify both containers are on the same Docker network.
- Check network configuration using docker network inspect.
- Verify container IPs and DNS resolution.
- Test connectivity using ping or curl.
- Check firewall rules.

2. A container cannot reach external services. What would you check first?

Answer:

- Verify internet connectivity from the container.
- Check DNS resolution.
- Inspect Docker bridge/NAT configuration.
- Review firewall and proxy settings.
- Validate host network connectivity.

3. Port mapping works locally but fails in production. How would you troubleshoot?

Answer:

- Verify container port exposure (docker ps).
 - Check host firewall/security groups.
 - Confirm application is listening on 0.0.0.0, not 127.0.0.1.
 - Validate reverse proxy/load balancer configuration.
 - Test using curl or telnet.
-

4. Docker bridge networking stops working. What would you investigate?

Answer:

- Check Docker daemon status.
 - Inspect bridge network (docker network inspect bridge).
 - Verify docker0 interface.
 - Check iptables/NAT rules.
 - Restart Docker service if necessary.
-

5. DNS resolution fails inside containers. How would you analyze it?

Answer:

- Check /etc/resolv.conf inside the container.
 - Test with nslookup or dig.
 - Verify Docker embedded DNS.
 - Check upstream DNS server availability.
 - Validate custom DNS configuration.
-

6. Overlay networking fails across hosts. How would you troubleshoot?

Answer:

- Verify swarm/Kubernetes cluster health.

- Check VXLAN ports (UDP 4789).
 - Confirm node connectivity.
 - Inspect overlay network configuration.
 - Review firewall rules.
-

7. Containers lose connectivity after restart. What would you investigate?

Answer:

- Verify network persistence.
 - Check Docker Compose or Kubernetes configuration.
 - Confirm service discovery.
 - Validate IP changes.
 - Inspect restart policies.
-

8. Firewall changes impact Docker networking. How would you troubleshoot?

Answer:

- Review recent firewall changes.
 - Verify Docker NAT rules.
 - Check required ports.
 - Test connectivity before and after firewall.
 - Restore missing rules if required.
-

9. Monitoring shows healthy containers but requests fail. What would you investigate?

Answer:

- Verify application health endpoints.
- Check reverse proxy routing.
- Validate service ports.
- Review application logs.

- Test container-to-container communication.
-

10. Network latency increases unexpectedly. How would you analyze it?

Answer:

- Check network utilization.
 - Measure latency using ping and traceroute.
 - Review CPU and memory usage.
 - Inspect packet loss.
 - Analyze Docker network performance.
-

11. Docker Compose networking behaves differently in production. Why?

Answer:

- Different network drivers.
 - Environment variable differences.
 - Compose vs Swarm/Kubernetes networking.
 - Different DNS behavior.
 - Firewall or proxy differences.
-

12. Container IP addresses change unexpectedly. How would you troubleshoot?

Answer:

- Understand that container IPs are ephemeral.
 - Use service names instead of IPs.
 - Verify network recreation.
 - Inspect Docker network configuration.
-

13. MTU mismatch causes intermittent failures. How would you investigate?

Answer:

- Check MTU values on host and containers.

- Test with varying packet sizes using ping -M do.
 - Compare overlay and physical network MTU.
 - Update Docker daemon MTU if needed.
-

14. Reverse proxy cannot reach backend containers. What would you troubleshoot?

Answer:

- Verify backend container is running.
 - Check Docker network membership.
 - Validate proxy configuration.
 - Confirm backend port exposure.
 - Test connectivity using curl.
-

15. Network changes impact multiple applications. How would you analyze them?

Answer:

- Identify recent network changes.
 - Review change records.
 - Check shared Docker networks.
 - Validate firewall/load balancer updates.
 - Roll back changes if required.
-

16. A Docker networking issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Docker daemon logs
- Container logs
- Network inspection output
- Firewall configuration
- Host network status

- Application logs
 - Deployment timeline
 - Monitoring dashboards
-

17. Which Docker commands are most useful?

Answer:

`docker ps`

`docker inspect`

`docker logs`

`docker exec`

`docker network ls`

`docker network inspect`

`docker stats`

`docker events`

`docker compose ps`

`docker compose logs`

18. What logs would you review first?

Answer:

- Docker daemon logs
- Container logs
- Reverse proxy logs (Nginx/Traefik)
- Application logs
- System logs (journalctl)
- Firewall logs
- Kubernetes logs (if applicable)

19. How would you validate networking changes before production?

Answer:

- Test in staging.
- Validate connectivity.
- Verify DNS resolution.
- Test port mappings.
- Perform smoke tests.
- Run rollback validation.

20. What controls reduce container networking risks?

Answer:

- Infrastructure as Code (IaC)
- Network policies
- Firewall rules
- Least-privilege access
- Automated testing
- Change approvals

- Monitoring and alerts
-

21. How would you improve Docker networking reliability?

Answer:

- Use user-defined networks.
 - Implement health checks.
 - Enable monitoring.
 - Use service discovery.
 - Standardize network configuration.
 - Automate deployments.
 - Document network architecture.
-

22. What testing should occur before releases?

Answer:

- Connectivity testing
 - DNS validation
 - Port mapping verification
 - Load testing
 - Failover testing
 - Smoke testing
 - Security testing
-

23. What should be documented in a Docker RCA?

Answer:

- Incident summary
- Timeline
- Root cause
- Affected containers/services

- Network changes
 - Resolution steps
 - Rollback actions
 - Lessons learned
 - Preventive measures
-

24. How would you improve network observability?

Answer:

- Enable centralized logging.
 - Monitor network metrics.
 - Use distributed tracing.
 - Implement dashboards.
 - Configure alerts for latency, packet loss, and failures.
 - Collect Docker events.
-

25. What preventive actions would reduce future incidents?

Answer:

- Standardize Docker network configuration.
- Automate network validation in CI/CD.
- Use health checks and readiness probes.
- Regularly review firewall and security policies.
- Perform disaster recovery drills.
- Enable continuous monitoring and alerting.
- Document network topology and runbooks.

Q: What is your Docker networking troubleshooting approach during a production incident?

Answer:

"I start by identifying the scope of the issue—whether it's affecting a single container, a service, or multiple hosts. I verify that the containers are running and attached to the

correct Docker network using docker ps and docker network inspect. I test connectivity between containers, check DNS resolution, validate port mappings, and review firewall, load balancer, and reverse proxy configurations. I also examine Docker daemon logs, container logs, and application logs to correlate failures. If a recent deployment or network change is responsible, I roll back to the last known good configuration. Once the service is restored, I perform a root cause analysis and implement preventive measures such as automated network validation, monitoring, standardized network policies, and documented recovery procedures."

1. HTTPS suddenly stops working in production. How would you investigate?

Answer:

- Verify certificate validity.
- Check web server/reverse proxy status.
- Test TLS handshake using openssl.
- Review SSL configuration.
- Check recent certificate or configuration changes.
- Verify Load Balancer and DNS.

2. SSL certificates expire unexpectedly. What would you check first?

Answer:

- Check certificate expiry date.
- Verify auto-renewal process (Let's Encrypt/Cert Manager).
- Review renewal logs.
- Check scheduled jobs.
- Validate certificate deployment.

3. Browsers report certificate mismatch errors. How would you troubleshoot?

Answer:

- Compare certificate CN/SAN with requested domain.
- Verify DNS points to the correct server.
- Check Load Balancer and reverse proxy.
- Ensure the correct certificate is installed.

4. Certificate renewal succeeds but applications still fail. Why?

Answer:

- Old certificate still cached.
- Service not restarted/reloaded.
- Wrong certificate chain.
- Incorrect certificate path.
- Load Balancer still serving old certificate.

5. Only one domain experiences SSL failures. How would you investigate?

Answer:

- Verify domain DNS.
- Check SNI configuration.
- Validate certificate mapping.
- Confirm virtual host configuration.
- Test using openssl s_client.

6. Load Balancer SSL termination stops working. How would you troubleshoot?

Answer:

- Verify certificate attached to the Load Balancer.
- Check listener configuration.
- Review Load Balancer logs.
- Validate backend health.
- Confirm recent configuration changes.

7. mTLS authentication fails after deployment. What would you analyze?

Answer:

- Verify client and server certificates.
- Check certificate trust chain.
- Validate CA configuration.
- Review certificate expiry.
- Inspect TLS handshake logs.

8. Monitoring shows healthy services but HTTPS requests fail. What would you investigate?

Answer:

- SSL certificate validity.
- Reverse proxy configuration.
- Load Balancer.
- Firewall.
- TLS handshake failures.
- DNS resolution.

9. Intermediate certificates are missing. How would you troubleshoot?

Answer:

- Verify full certificate chain.
- Install intermediate certificates.
- Test using openssl.
- Validate browser trust.

10. Certificate rotation causes downtime. What would you investigate?

Answer:

- Compare old and new certificates.
- Check deployment timing.
- Verify certificate permissions.
- Confirm service reload.
- Validate automation.

11. TLS handshake latency increases unexpectedly. How would you analyze it?

Answer:

- Measure TLS handshake time.
- Check CPU utilization.
- Verify certificate size and chain.
- Review network latency.

- Analyze Load Balancer performance.

12. Reverse proxy rejects secure connections. What would you check?

Answer:

- Certificate paths.
- TLS configuration.
- Supported protocols/ciphers.
- Proxy logs.
- Backend connectivity.

13. Applications work internally but fail externally. How would you investigate?

Answer:

- Check DNS.
- Verify firewall/security groups.
- Inspect Load Balancer.
- Confirm SSL termination.
- Validate public certificate.

14. Security scans report invalid certificates. What would you do?

Answer:

- Verify certificate chain.
- Check expiry.
- Validate hostname.
- Replace invalid certificate.
- Retest using security scanners.

15. A rollback restores HTTPS immediately. What evidence would you collect?

Answer:

- Deployment history.
- Certificate versions.
- Reverse proxy configuration.
- Load Balancer changes.

- TLS logs.
- Git commits.
- Timeline of events.

16. An SSL issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Certificate details.
- Expiry information.
- TLS handshake logs.
- Web server logs.
- Load Balancer logs.
- Reverse proxy logs.
- Monitoring metrics.
- Deployment timeline.
- Recent configuration changes.

17. Which metrics help identify certificate failures?

Answer:

- Certificate expiry days.
- TLS handshake success rate.
- HTTPS response codes.
- TLS handshake latency.
- SSL error count.
- Connection failures.
- Certificate renewal status.

18. What logs would you review first?

Answer:

- Nginx/Apache logs.
- Load Balancer logs.
- Cert-Manager logs.

- Application logs.
- Kubernetes Ingress logs.
- System logs (journalctl).

19. How would you validate certificate changes before production?

Answer:

- Test in staging.
- Verify certificate chain.
- Confirm hostname matching.
- Test TLS handshake.
- Validate browser compatibility.
- Perform smoke tests.

20. What controls reduce certificate risks?

Answer:

- Automated certificate renewal.
- Certificate monitoring.
- Infrastructure as Code.
- Change approvals.
- RBAC.
- Secret management.
- Short-lived certificates.

21. How would you improve certificate lifecycle management?

Answer:

- Automate issuance and renewal.
- Use Cert-Manager or cloud certificate services.
- Centralize certificate inventory.
- Track expiry dates.
- Standardize certificate deployment.

22. What testing should occur before renewals?

Answer:

- Renewal testing.
- Certificate chain validation.
- Browser compatibility.
- TLS handshake testing.
- Load Balancer validation.
- Application smoke tests.

23. What should be documented in an SSL RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- Affected domains.
- Certificate details.
- Resolution steps.
- Rollback actions.
- Lessons learned.
- Preventive actions.

24. How would you improve certificate monitoring?

Answer:

- Monitor expiry dates.
- Alert before expiration.
- Track TLS handshake failures.
- Monitor certificate renewals.
- Centralize dashboards.
- Enable synthetic HTTPS monitoring.

25. What preventive actions would reduce future incidents?

Answer:

- Automate certificate renewal.
- Enable expiry alerts (30/15/7 days).
- Regularly validate certificate chains.
- Test renewals in staging.
- Use centralized certificate management.
- Document renewal procedures.
- Perform periodic certificate audits.
- Monitor TLS health continuously.
- Maintain rollback procedures for certificate changes.

Q: What is your approach to troubleshooting SSL/TLS issues in production?

Answer:

"I begin by confirming the scope of the issue and verifying the certificate's validity, expiry date, Common Name (CN), and Subject Alternative Names (SAN). I test the TLS handshake using tools like openssl s_client or curl -v and review web server, reverse proxy, Load Balancer, and certificate management logs. I verify that the correct certificate chain, including intermediate certificates, is installed and that DNS and SNI are correctly configured. If a recent deployment or certificate rotation caused the issue, I compare configurations and roll back if necessary to restore service. After recovery, I perform a root cause analysis, automate certificate lifecycle management, implement expiry monitoring and alerts, and validate future renewals in a staging environment before production."

Check certificate details

```
openssl x509 -in cert.pem -text -noout
```

Test HTTPS endpoint

```
openssl s_client -connect example.com:443
```

View certificate expiry

```
openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -dates
```

Test HTTPS request

```
curl -v https://example.com
```

Check certificate chain

```
openssl verify -CAfile ca_bundle.pem cert.pem
```

View Nginx logs

```
tail -f /var/log/nginx/error.log
```



```
# View Apache logs
tail -f /var/log/httpd/error_log

# Check Kubernetes Ingress
kubectl describe ingress <ingress-name>

# View TLS Secret
kubectl get secret <tls-secret> -o yaml
```

1. Terraform state file becomes corrupted. How would you investigate?

Answer:

- Verify backend accessibility.
- Check state file integrity and version history.
- Restore from backend versioning/backup if available.
- Compare infrastructure with the last valid state.
- Run terraform plan after recovery.

2. State locking fails during deployment. What would you check?

Answer:

- Verify backend lock service (DynamoDB/Azure Blob/GCS).
- Check for another active Terraform operation.
- Inspect stale locks.
- Release the lock only after confirming no active deployment.

3. Resources exist in AWS but not in Terraform state. How would you troubleshoot?

Answer:

- Verify whether resources were created manually.
- Compare AWS resources with Terraform state.
- Import missing resources using terraform import.
- Run terraform plan to validate consistency.

4. Multiple engineers modify the same state file. How would you investigate?

Answer:

- Check backend lock history.
- Review CI/CD pipeline execution.

- Verify audit logs.
- Ensure remote backend with state locking is used.
- Restrict direct state modifications.

5. Terraform plans unexpected resource deletions. What would you analyze?

Answer:

- Review terraform plan.
- Compare current state with configuration.
- Check variable changes.
- Verify provider version.
- Investigate state drift before applying.

6. Backend migration causes deployment failures. How would you troubleshoot?

Answer:

- Verify backend configuration.
- Confirm state migration completed successfully.
- Check backend credentials and permissions.
- Compare migrated state with original state.
- Reinitialize using terraform init.

7. Remote state access fails. What would you investigate?

Answer:

- Verify backend connectivity.
- Check IAM/RBAC permissions.
- Validate backend configuration.
- Review storage availability.
- Confirm authentication credentials.

8. State drift is detected after production changes. How would you analyze it?

Answer:

- Compare Terraform state with live infrastructure.
- Identify manual changes.

- Run terraform plan.
- Import unmanaged resources if needed.
- Enforce IaC-only changes.

9. Monitoring shows healthy infrastructure but deployments fail. What would you check?

Answer:

- Check Terraform backend.
- Review pipeline logs.
- Validate provider authentication.
- Inspect state locking.
- Verify recent configuration changes.

10. State file version mismatch causes errors. How would you troubleshoot?

Answer:

- Check Terraform CLI version.
- Verify backend state version.
- Upgrade providers if needed.
- Run terraform init -upgrade.
- Validate compatibility.

11. A rollback requires state recovery. How would you investigate?

Answer:

- Identify last known good state.
- Restore backend version if available.
- Verify infrastructure consistency.
- Run terraform plan before applying.

12. Terraform imports fail unexpectedly. What would you analyze?

Answer:

- Verify resource ID.
- Check provider configuration.

- Confirm resource exists.
- Validate permissions.
- Review import syntax and logs.

13. Infrastructure changes occur outside Terraform. How would you respond?

Answer:

- Identify manual changes.
- Assess business impact.
- Import or revert changes.
- Update Terraform configuration.
- Enforce change management and RBAC.

14. Pipeline execution fails because of state conflicts. How would you troubleshoot?

Answer:

- Check active state lock.
- Review concurrent pipeline executions.
- Inspect backend logs.
- Retry after lock release.
- Serialize deployments if required.

15. One incorrect apply impacts multiple environments. How would you investigate?

Answer:

- Verify workspace/environment.
- Review variables and backend configuration.
- Identify affected resources.
- Roll back if possible.
- Separate state files per environment.

16. A Terraform state issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Terraform state file
- Terraform plan/output
- Backend logs
- CI/CD logs
- Audit logs
- Provider logs
- Infrastructure changes
- Timeline
- Recent commits

17. Which Terraform logs are most useful?

Answer:

- Terraform CLI logs (TF_LOG)
- CI/CD pipeline logs
- Backend storage logs
- Provider logs
- Cloud audit logs (CloudTrail/Azure Activity/GCP Audit)
- State lock logs

18. What commands would you execute first?

Answer:

terraform version
 terraform init
 terraform validate
 terraform plan
 terraform state list
 terraform state show <resource>
 terraform show
 terraform providers
 terraform workspace show

19. How would you validate state before production?

Answer:

- Run terraform validate.
- Review terraform plan.
- Verify backend and workspace.
- Confirm state locking.
- Peer review infrastructure changes.
- Test in staging.

20. What controls reduce IaC risks?

Answer:

- Remote state backend.
- State locking.
- Version control.
- Code reviews.
- CI/CD validation.
- RBAC.
- Policy as Code (OPA/Sentinel).
- Automated testing.

21. How would you improve Terraform reliability?

Answer:

- Use remote backend with locking.
- Separate state per environment.
- Modularize code.
- Automate validation.
- Enable drift detection.
- Version providers.
- Monitor deployments.

22. What testing should occur before backend changes?

Answer:

- State backup verification.

- Backend migration testing.
- Restore testing.
- terraform plan validation.
- Permission testing.
- Rollback testing.

23. What should be documented in a Terraform RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- State/backend details.
- Affected resources.
- Resolution.
- Rollback actions.
- Lessons learned.
- Preventive actions.

24. How would you improve infrastructure governance?

Answer:

- Enforce Infrastructure as Code.
- Branch protection and approvals.
- Policy as Code.
- Environment separation.
- Drift monitoring.
- Regular audits.
- Centralized state management.

25. What preventive actions would reduce future incidents?

Answer:

- Use remote state with locking.

- Enable backend versioning.
- Restrict manual infrastructure changes.
- Run terraform plan in CI/CD.
- Review all changes through pull requests.
- Perform regular drift detection.
- Test backend migrations.
- Maintain state backups.
- Separate environments with dedicated state files.
- Document rollback procedures.

Q: What is your Terraform state troubleshooting approach during a production incident?

Answer:

"I first determine whether the issue is related to the state file, backend, or the actual infrastructure. I verify backend connectivity, state locking, and inspect the current state using terraform state list and terraform show. I compare the Terraform state with the live infrastructure to identify drift or missing resources and review recent pipeline executions, provider changes, and manual infrastructure modifications. If the issue is caused by state corruption or an incorrect update, I restore the last known good state from backend versioning or backups, validate it with terraform plan, and apply changes only after confirming no unexpected modifications. After restoring service, I perform a root cause analysis and strengthen governance through remote state, locking, code reviews, automated validation, and regular drift detection."

Initialize Terraform

terraform init

Validate configuration

terraform validate

Preview infrastructure changes

terraform plan

Apply changes

terraform apply

Display current state

terraform show

List managed resources

terraform state list

Show details of a resource

terraform state show <resource>

Import an existing resource

terraform import <resource_address> <resource_id>

Refresh state

terraform refresh

Check current workspace

terraform workspace show

View provider versions

terraform providers

1. Users report issues before monitoring detects them. How would you investigate?

Answer:

- Verify monitoring coverage.
- Check alert thresholds.
- Review logs, metrics, and traces.
- Compare user-reported timestamps with telemetry.
- Identify missing instrumentation and improve alerts.

2. Metrics are available but logs are missing. What would you check?

Answer:

- Verify logging agent (Fluentd/Fluent Bit/Promtail).
- Check log pipeline health.
- Validate application logging configuration.
- Review storage availability.
- Confirm log retention policies.

3. Traces stop appearing during a Sev-1 incident. How would you troubleshoot?

Answer:

- Check tracing agent/collector.
- Verify OpenTelemetry/Jaeger/Zipkin services.

- Review exporter configuration.
- Check network connectivity.
- Validate sampling configuration.

4. Dashboards remain green while customers face downtime. How would you investigate?

Answer:

- Verify dashboard metrics reflect customer experience.
- Check synthetic monitoring.
- Review application logs.
- Validate health checks.
- Compare business metrics with infrastructure metrics.

5. Alerts trigger too late during outages. How would you analyze them?

Answer:

- Review alert thresholds.
- Analyze evaluation intervals.
- Check alert rules.
- Compare alert timestamps with incident timeline.
- Tune thresholds for earlier detection.

6. Logs, metrics and traces show conflicting information. How would you proceed?

Answer:

- Correlate using timestamps.
- Verify time synchronization (NTP).
- Compare telemetry sources.
- Prioritize end-to-end traces.
- Review application logs for confirmation.

7. Alert fatigue causes engineers to ignore notifications. How would you improve it?

Answer:

- Remove noisy alerts.

- Prioritize critical alerts.
- Reduce duplicate notifications.
- Implement alert correlation.
- Regularly review alert effectiveness.

8. A monitoring upgrade impacts observability. How would you troubleshoot?

Answer:

- Compare old and new configurations.
- Verify agents are running.
- Review upgrade logs.
- Validate dashboards and alerts.
- Roll back if required.

9. Root cause cannot be identified because of missing telemetry. What would you do?

Answer:

- Collect available logs and metrics.
- Enable additional instrumentation.
- Reproduce the issue in staging.
- Add missing tracing and logging.
- Document telemetry gaps.

10. Application latency increases without alerts. How would you investigate?

Answer:

- Review latency dashboards.
- Check APM traces.
- Analyze database performance.
- Review GC, CPU, and memory.
- Update latency alert thresholds.

11. Different monitoring tools report different values. How would you analyze them?

Answer:

- Compare collection intervals.
- Verify scrape frequency.
- Check aggregation methods.
- Validate time synchronization.
- Compare raw metrics.

12. Business KPIs remain healthy while infrastructure degrades. What would you investigate?

Answer:

- Determine if degradation has reached users.
- Review infrastructure capacity.
- Check redundancy.
- Monitor trends before business impact occurs.

13. Monitoring agents stop reporting data. How would you troubleshoot?

Answer:

- Verify agent status.
- Check connectivity.
- Review agent logs.
- Validate authentication.
- Restart agent if required.

14. Teams cannot correlate events across services. How would you improve it?

Answer:

- Implement distributed tracing.
- Use correlation/request IDs.
- Centralize logs.
- Standardize timestamps.
- Create shared dashboards.

15. Observability gaps delay incident resolution. How would you address them?

Answer:

- Improve instrumentation.
- Increase telemetry coverage.
- Add synthetic monitoring.
- Standardize logging.
- Expand dashboards and alerts.

16. An observability issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Monitoring dashboards
- Alert history
- Logs
- Metrics
- Distributed traces
- Incident timeline
- Deployment history
- Configuration changes
- Customer impact

17. Which telemetry is most valuable during incidents?

Answer:

- Logs
- Metrics
- Distributed traces
- Events
- Application Performance Monitoring (APM)
- Kubernetes events
- Infrastructure metrics
- Business KPIs

18. What dashboards would you check first?

Answer:

- Application Health
- Error Rate
- Latency
- CPU & Memory
- Kubernetes Cluster
- Network
- Database
- Load Balancer
- Business KPI Dashboard

19. How would you validate monitoring before production?

Answer:

- Test alerts.
- Verify dashboards.
- Validate log collection.
- Confirm trace generation.
- Perform synthetic transactions.
- Test notification channels.

20. What controls reduce observability gaps?

Answer:

- Standardized instrumentation.
- Centralized logging.
- Distributed tracing.
- Alert reviews.
- Monitoring as Code.
- SLO/SLI implementation.
- Change management.

21. How would you improve incident detection?

Answer:

- Better alert thresholds.
- Synthetic monitoring.
- APM integration.
- Business KPI monitoring.
- Correlated alerts.
- AI/ML anomaly detection.
- Continuous tuning.

22. What testing should occur before monitoring changes?

Answer:

- Alert testing.
- Dashboard validation.
- Log ingestion testing.
- Trace validation.
- Failover testing.
- Notification testing.

23. What should be documented in an Observability RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- Missing telemetry.
- Alert performance.
- Resolution.
- Lessons learned.
- Monitoring improvements.
- Preventive actions.

24. How would you improve production visibility?

Answer:

- Centralize logs.
- Enable distributed tracing.
- Build real-time dashboards.
- Monitor SLIs/SLOs.
- Add synthetic monitoring.
- Improve application instrumentation.
- Use correlation IDs.

25. What preventive actions would reduce future incidents?

Answer:

- Implement the three pillars of observability (Logs, Metrics, Traces).
- Define SLIs, SLOs, and error budgets.
- Tune alerts to reduce noise.
- Continuously improve dashboards.
- Standardize instrumentation.
- Perform regular observability reviews.
- Test monitoring after every release.
- Automate alert validation.
- Conduct incident drills.
- Review telemetry coverage regularly.

Q: What is your observability troubleshooting approach during a production incident?

Answer:

"I start by determining the scope and impact of the incident using monitoring dashboards and customer reports. I correlate the three pillars of observability—metrics, logs, and distributed traces—to understand what changed and when. I check key indicators such as latency, error rate, traffic, and resource utilization, then drill into application and infrastructure logs while following request traces across services. I also review recent deployments and configuration changes to identify potential causes. Once the issue is mitigated, I perform a root cause analysis, identify any telemetry or

alerting gaps, improve dashboards and thresholds, and ensure better observability coverage to detect similar issues earlier in the future.”

Kubernetes events

kubectl get events --sort-by=.metadata.creationTimestamp

Pod logs

kubectl logs <pod-name>

Previous container logs

kubectl logs <pod-name> --previous

Resource usage

kubectl top pod

kubectl top node

Check Prometheus targets

curl http://<prometheus>:9090/api/v1/targets

Test application endpoint

curl -v <https://app.example.com/health>

View system logs

journalctl -xe

View Docker logs

docker logs <container>

Network connectivity

ping <host>

traceroute <host>

DNS resolution

nslookup <hostname>

1. New worker nodes fail to join the EKS cluster. How would you investigate?

Answer:

- Verify node IAM role.
- Check aws-auth ConfigMap or EKS Access Entry.
- Validate bootstrap/user-data script.
- Check Security Groups and VPC networking.
- Review kubelet logs.

2. Managed Node Groups become unhealthy after an upgrade. What would you check first?

Answer:

- Review EKS upgrade events.
- Check node status (kubectl get nodes).
- Inspect kubelet logs.
- Verify AMI compatibility.
- Review AWS Health and EKS events.

3. Auto Scaling launches nodes but pods remain Pending. How would you troubleshoot?

Answer:

- Check node registration.
- Verify node labels and taints.
- Review scheduler events.
- Check CPU/Memory availability.
- Validate Cluster Autoscaler configuration.

4. Node replacement causes application downtime. How would you investigate?

Answer:

- Check Pod Disruption Budgets (PDBs).
- Verify drain process.
- Review Deployment strategy.
- Confirm readiness/liveness probes.
- Ensure replicas are distributed across nodes.

5. Worker nodes cannot communicate with the API Server. What would you analyze?

Answer:

- Verify API endpoint accessibility.
- Check Security Groups/NACLs.
- Validate Route Tables.

- Review kubelet configuration.
- Test network connectivity.

6. Nodes repeatedly enter NotReady state. How would you troubleshoot?

Answer:

- Check kubelet logs.
- Review node conditions.
- Verify container runtime.
- Check disk, CPU, and memory.
- Inspect network connectivity to the control plane.

7. IAM Role changes prevent nodes from joining the cluster. How would you investigate?

Answer:

- Verify node IAM role policies.
- Check AmazonEKSWorkerNodePolicy.
- Validate ECR and CNI permissions.
- Review IAM trust relationship.
- Confirm aws-auth mapping or access entries.

8. Monitoring shows healthy nodes but workloads fail. What would you check?

Answer:

- Verify pod status.
- Check application logs.
- Review Services and Endpoints.
- Validate Ingress/Load Balancer.
- Check DNS and network policies.

9. New nodes fail after a Launch Template update. How would you troubleshoot?

Answer:

- Compare old vs new Launch Template.
- Verify AMI ID.

- Validate user-data/bootstrap script.
- Check IAM profile.
- Review Security Groups.

10. Node Groups scale successfully but applications remain unavailable. Why?

Answer:

- Pods not scheduled.
- Readiness probe failures.
- Missing labels/taints.
- Network issues.
- Application startup failures.

11. Spot Node Groups terminate unexpectedly. How would you investigate?

Answer:

- Check EC2 Spot interruption notices.
- Review Auto Scaling events.
- Verify Cluster Autoscaler.
- Ensure workloads are spread across On-Demand and Spot nodes.

12. Security Group updates impact node communication. What would you troubleshoot?

Answer:

- Verify inbound/outbound rules.
- Check API Server access.
- Validate node-to-node communication.
- Review Load Balancer rules.
- Confirm VPC routing.

13. Multiple Node Groups fail after cluster upgrades. How would you prioritize?

Answer:

- Prioritize production workloads.
- Identify common root cause.

- Compare Node Group configurations.
- Review upgrade compatibility.
- Restore critical services first.

14. Rollback restores node stability. What evidence would you collect?

Answer:

- Upgrade history.
- Launch Template versions.
- Node events.
- Deployment timeline.
- IAM changes.
- Security Group changes.
- Application logs.

15. Node Group updates increase deployment failures. How would you analyze them?

Answer:

- Compare old vs new configuration.
- Review scheduler events.
- Check node capacity.
- Validate Kubernetes version compatibility.
- Review rollout history.

16. An EKS Node Group issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Node events
- EKS events
- Auto Scaling logs
- CloudTrail logs
- kubelet logs
- Deployment history

- Cluster Autoscaler logs
- Application logs
- Timeline
- Recent infrastructure changes

17. Which EKS metrics are most useful?

Answer:

- Node Ready Status
- Node CPU & Memory
- Pod Pending Count
- Pod Restart Count
- API Server Latency
- Cluster Autoscaler Metrics
- Node Network Errors
- Disk Pressure
- Pod Scheduling Latency

18. What logs would you review first?

Answer:

- kubelet logs
- Cluster Autoscaler logs
- CloudTrail logs
- EKS Control Plane logs
- Application logs
- VPC Flow Logs
- EC2 System Logs

19. How would you validate Node Group changes before production?

Answer:

- Test in staging.
- Validate node join process.

- Verify workloads schedule correctly.
- Perform rolling update tests.
- Test node drain and replacement.
- Validate monitoring and alerts.

20. What controls reduce cluster risks?

Answer:

- Infrastructure as Code (Terraform)
- Managed Node Groups
- Pod Disruption Budgets
- RBAC/IAM least privilege
- Automated testing
- Cluster Autoscaler
- Version control and approvals

21. How would you improve EKS reliability?

Answer:

- Multi-AZ Node Groups.
- Mix On-Demand and Spot nodes.
- Configure Cluster Autoscaler.
- Enable managed upgrades.
- Use Pod Disruption Budgets.
- Monitor cluster health.
- Regular capacity planning.

22. What testing should occur before upgrades?

Answer:

- Kubernetes version compatibility.
- Node Group upgrade testing.
- Application smoke tests.
- Load testing.

- Rollback testing.
- Node drain validation.
- Disaster recovery testing.

23. What should be documented in an EKS RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- Node Group details.
- Cluster version.
- Impacted applications.
- Resolution.
- Rollback actions.
- Lessons learned.
- Preventive actions.

24. How would you improve cluster observability?

Answer:

- Enable EKS Control Plane logs.
- Monitor node and pod metrics.
- Centralize application logs.
- Use distributed tracing.
- Build Grafana dashboards.
- Configure proactive alerts.

25. What preventive actions would reduce future incidents?

Answer:

- Validate upgrades in staging.
- Use managed node groups.
- Enable automated health monitoring.

- Maintain Pod Disruption Budgets.
- Test node replacement regularly.
- Keep IAM policies under version control.
- Enable cluster autoscaling with monitoring.
- Perform regular capacity planning.
- Conduct periodic disaster recovery drills.
- Document upgrade and rollback procedures.

Q: What is your approach to troubleshooting EKS Node Group issues in production?

Answer:

"I first determine whether the issue is with node provisioning, node registration, scheduling, or application availability. I verify that new EC2 instances are launched successfully, then check whether they join the cluster using `kubectl get nodes`. If they don't, I review the node IAM role, bootstrap script, aws-auth configuration or EKS access entries, Security Groups, VPC connectivity, and kubelet logs. If nodes are healthy but workloads are failing, I inspect pod events, scheduler decisions, taints, tolerations, and resource availability. I also review recent upgrades, Launch Template changes, and Auto Scaling events. After restoring service, I perform a root cause analysis and strengthen reliability through staged upgrades, Infrastructure as Code, automated validation, Pod Disruption Budgets, monitoring, and documented rollback procedures."

Check node status

```
kubectl get nodes -o wide
```

Describe a node

```
kubectl describe node <node-name>
```

View pod scheduling issues

```
kubectl get pods -A
```

```
kubectl describe pod <pod-name>
```

Check cluster events

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Resource utilization

```
kubectl top nodes
```

```
kubectl top pods
```

Review deployment rollout

```
kubectl rollout status deployment <deployment-name>
```

```
kubectl rollout history deployment <deployment-name>
```

```
# AWS CLI - Describe Node Group
aws eks describe-nodegroup \
  --cluster-name <cluster-name> \
  --nodegroup-name <nodegroup-name>

# Auto Scaling activity
aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name <asg-name>

# EC2 instance status
aws ec2 describe-instance-status
```

1. An EC2 instance suddenly loses access to S3. How would you investigate?

Answer:

- Verify the attached IAM Role.
- Check S3 bucket policy.
- Validate IAM permissions.
- Confirm STS credentials are valid.
- Review CloudTrail for AccessDenied.
- Test connectivity using AWS CLI.

2. A pod using IRSA cannot access AWS services. What would you check first?

Answer:

- Verify ServiceAccount annotation.
- Check IAM Role trust policy.
- Validate OIDC provider.
- Confirm IAM permissions.
- Review pod logs and STS AssumeRoleWithWebIdentity errors.

3. AssumeRole requests start failing after deployment. How would you troubleshoot?

Answer:

- Check IAM trust relationship.
- Verify role ARN.
- Validate STS endpoint access.

- Review CloudTrail logs.
- Confirm no recent IAM policy changes.

4. Lambda works in staging but fails in production due to IAM. What would you investigate?

Answer:

- Compare execution roles.
- Review IAM policies.
- Check environment variables.
- Verify resource-based policies.
- Review CloudWatch and CloudTrail logs.

5. Temporary credentials expire unexpectedly. How would you analyze the issue?

Answer:

- Check STS session duration.
- Verify credential refresh.
- Review application SDK configuration.
- Validate IAM Role session limits.
- Check system time synchronization.

6. IAM Role permissions are modified accidentally. How would you troubleshoot?

Answer:

- Review CloudTrail for policy changes.
- Compare with previous policy version.
- Restore the last known good policy.
- Validate access after rollback.
- Restrict future policy changes.

7. Cross-account access suddenly stops working. What would you investigate?

Answer:

- Verify trust policy.
- Check AssumeRole permissions.

- Validate external ID (if used).
- Review SCPs (Service Control Policies).
- Analyze CloudTrail logs.

8. CloudTrail shows AccessDenied events. What would you check?

Answer:

- Review denied API calls.
- Verify IAM policies.
- Check explicit denies.
- Validate SCPs and permission boundaries.
- Review resource policies.

9. ECS tasks cannot access Secrets Manager. How would you troubleshoot?

Answer:

- Verify Task IAM Role.
- Check Secrets Manager permissions.
- Validate secret ARN.
- Review ECS task logs.
- Confirm KMS permissions if the secret is encrypted.

10. Monitoring shows healthy infrastructure but AWS API calls fail. What would you investigate?

Answer:

- Verify IAM permissions.
- Check STS authentication.
- Review API rate limits.
- Validate SDK credentials.
- Analyze CloudTrail logs.

11. A rollback restores access immediately. What evidence would you collect?

Answer:

- IAM policy versions.

- CloudTrail events.
- Deployment history.
- Git commits.
- Application logs.
- STS logs.
- Timeline of changes.

12. IAM policies appear correct but requests still fail. How would you troubleshoot?

Answer:

- Check resource-based policies.
- Review explicit denies.
- Validate SCPs.
- Verify permission boundaries.
- Test using IAM Policy Simulator.

13. STS requests fail intermittently. What would you investigate?

Answer:

- Check network connectivity.
- Verify STS endpoint.
- Review credential refresh.
- Analyze CloudTrail logs.
- Check AWS service health.

14. Multiple applications lose AWS access simultaneously. How would you prioritize?

Answer:

- Identify shared IAM Roles.
- Check organization-wide policy changes.
- Review STS and CloudTrail logs.
- Restore access for production-critical services first.
- Communicate incident status.

15. A policy update impacts production. How would you analyze the impact?

Answer:

- Compare old and new policy.
- Identify affected roles.
- Review AccessDenied events.
- Roll back if necessary.
- Validate applications after recovery.

16. An IAM Role issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- CloudTrail logs
- IAM policy versions
- Role trust policies
- STS logs
- AccessDenied events
- Deployment history
- CloudWatch logs
- Timeline
- Recent IAM changes

17. Which AWS logs are most useful during IAM incidents?

Answer:

- CloudTrail
- CloudWatch Logs
- AWS Config
- IAM Access Analyzer
- VPC Flow Logs (if network access is involved)
- STS logs (via CloudTrail)

18. What metrics indicate authentication failures?

Answer:

- AccessDenied count
- STS AssumeRole failures
- AWS API error rate
- Authentication failure rate
- CloudWatch error metrics
- 4xx API responses

19. How would you validate IAM Role changes before production?

Answer:

- Test in a non-production environment.
- Use IAM Policy Simulator.
- Perform least-privilege validation.
- Verify AssumeRole functionality.
- Execute application smoke tests.

20. What controls reduce permission-related risks?

Answer:

- Least privilege.
- IAM Roles instead of long-lived keys.
- Multi-factor authentication (MFA).
- Policy reviews.
- RBAC.
- SCPs.
- Infrastructure as Code (Terraform/CloudFormation).

21. How would you improve IAM governance?

Answer:

- Centralize IAM management.
- Use role-based access.
- Enable AWS Config.
- Perform periodic access reviews.

- Rotate credentials automatically.
- Enable IAM Access Analyzer.

22. What testing should occur before policy updates?

Answer:

- IAM Policy Simulator testing.
- Application smoke testing.
- Cross-account access validation.
- IRSA/ECS/Lambda role testing.
- Rollback testing.

23. What should be documented in an IAM RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- Affected roles/policies.
- AccessDenied events.
- Resolution.
- Rollback actions.
- Lessons learned.
- Preventive actions.

24. How would you improve access visibility?

Answer:

- Enable CloudTrail organization-wide.
- Use IAM Access Analyzer.
- Monitor AccessDenied events.
- Create CloudWatch dashboards.
- Regularly review IAM changes.
- Audit unused permissions.

25. What preventive actions would reduce future incidents?

Answer:

- Enforce least-privilege access.
- Manage IAM through Infrastructure as Code.
- Enable CloudTrail and AWS Config.
- Review IAM policies regularly.
- Test policy changes before production.
- Use IAM Access Analyzer.
- Monitor STS and AccessDenied events.
- Enable automated alerts.
- Perform periodic permission audits.
- Maintain rollback procedures for IAM changes.

Q: What is your approach to troubleshooting IAM Role issues in production?

Answer:

"I first determine whether the issue is related to authentication, authorization, or role assumption. I verify that the correct IAM Role is attached and review both the identity-based policy and any resource-based policies. I check the role's trust relationship, STS AssumeRole or AssumeRoleWithWebIdentity operations, and analyze CloudTrail for AccessDenied events. If the workload uses IRSA, ECS Task Roles, or Lambda execution roles, I validate the corresponding configuration and permissions. I also compare recent IAM changes, deployments, and organization-level controls such as Service Control Policies (SCPs). If a recent policy update caused the issue, I roll back to the last known working version, restore service, and then perform a root cause analysis. To prevent recurrence, I enforce least privilege, manage IAM through Infrastructure as Code, validate policy changes in non-production environments, and continuously monitor IAM-related events and authentication failures."

Check caller identity

```
aws sts get-caller-identity
```

Assume an IAM Role

```
aws sts assume-role \
```

```
--role-arn arn:aws:iam::<account-id>:role/<role-name> \
```

```
--role-session-name test-session
```

```
# Test S3 access
aws s3 ls s3://<bucket-name>

# Get IAM Role details
aws iam get-role --role-name <role-name>

# List attached policies
aws iam list-attached-role-policies --role-name <role-name>

# Simulate IAM policy
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::<account-id>:role/<role-name> \
  --action-names s3:GetObject

# View EKS ServiceAccount (IRSA)
kubectl describe sa <service-account>

# Check ECS Task definition
aws ecs describe-task-definition --task-definition <task-def>
```

1. Applications suddenly lose database connectivity. How would you investigate?

Answer:

- Verify database availability.
- Test network connectivity.
- Check Security Groups/Firewall.
- Validate database endpoint and DNS.
- Review application and database logs.
- Check recent deployments or configuration changes.

2. Connection pools are exhausted. What would you check first?

Answer:

- Check active connections.
- Verify connection pool configuration.
- Look for connection leaks.
- Review slow queries.
- Check application thread usage.

3. Database authentication starts failing. How would you troubleshoot?

Answer:

- Verify username/password or IAM authentication.
- Check secret rotation.
- Validate database user permissions.
- Review authentication logs.
- Confirm SSL requirements.

4. Only one application cannot connect to the database. How would you investigate?

Answer:

- Compare application configuration with working apps.
- Verify credentials.
- Check network path.
- Validate DNS resolution.
- Review application logs.

5. Read replica connectivity stops working. What would you analyze?

Answer:

- Verify replica health.
- Check replication status.
- Validate replica endpoint.
- Review network and Security Groups.
- Confirm application read/write configuration.

6. SSL database connections fail after certificate updates. How would you troubleshoot?

Answer:

- Verify client trusts the new CA.
- Check certificate expiry.
- Validate SSL mode.
- Confirm certificate chain.
- Restart applications if required.

7. Connection timeouts increase during peak traffic. What would you investigate?

Answer:

- Check database CPU and memory.
- Review connection pool usage.
- Analyze slow queries.
- Verify network latency.
- Check database connection limits.

8. Security Group changes impact database access. How would you troubleshoot?

Answer:

- Compare Security Group rules.
- Verify inbound/outbound ports (e.g., 3306/5432).
- Check NACLs.
- Test connectivity using telnet or nc.
- Roll back incorrect rules if needed.

9. Monitoring shows healthy databases but applications fail. What would you investigate?

Answer:

- Review application logs.
- Validate connection strings.
- Check authentication.
- Verify connection pool health.
- Confirm DNS resolution.

10. DNS resolution affects database connectivity. How would you analyze it?

Answer:

- Test with nslookup or dig.
- Verify database endpoint.
- Check DNS cache.
- Confirm Route53/private DNS configuration.

- Test using the database IP if appropriate.

11. Database failover breaks application connectivity. How would you troubleshoot?

Answer:

- Verify new primary endpoint.
- Check DNS propagation.
- Validate application reconnect logic.
- Review failover events.
- Confirm connection pool refresh.

12. Connection latency increases unexpectedly. What would you investigate?

Answer:

- Check network latency.
- Review database performance.
- Analyze slow queries.
- Verify connection pool utilization.
- Monitor CPU and I/O.

13. Database parameters change after maintenance. How would you analyze them?

Answer:

- Compare parameter groups.
- Review maintenance logs.
- Check performance impact.
- Validate compatibility.
- Roll back changes if required.

14. A deployment introduces connection failures. How would you troubleshoot?

Answer:

- Compare old and new configuration.
- Verify secrets/configuration.
- Check environment variables.

- Review deployment logs.
- Roll back if necessary.

15. Multiple applications lose database access simultaneously. How would you prioritize?

Answer:

- Confirm database availability.
- Identify shared dependencies.
- Prioritize production-critical applications.
- Review infrastructure changes.
- Restore connectivity first, then investigate root cause.

16. A database connectivity issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Application logs
- Database logs
- Connection pool metrics
- Database performance metrics
- Network connectivity results
- Security Group changes
- DNS resolution results
- Failover events
- Deployment history
- Timeline of the incident

17. Which database metrics are most useful?

Answer:

- Active Connections
- Connection Pool Usage
- Query Latency
- Slow Queries

- CPU Utilization
- Memory Utilization
- Disk I/O
- Replication Lag
- Connection Errors
- Deadlocks

18. What logs would you review first?

Answer:

- Application logs
- Database error logs
- Slow query logs
- Audit logs
- Connection logs
- CloudWatch/RDS logs
- Kubernetes pod logs (if applicable)

19. How would you validate connectivity before production?

Answer:

- Test database connectivity.
- Validate credentials.
- Verify SSL connectivity.
- Run application smoke tests.
- Test failover.
- Verify connection pooling.

20. What controls reduce database access risks?

Answer:

- Least-privilege database users.
- Secret management (AWS Secrets Manager/Vault).
- IAM authentication where supported.

- Security Groups.
- SSL/TLS encryption.
- Connection pooling.
- Infrastructure as Code.

21. How would you improve application resilience?

Answer:

- Configure connection retries.
- Tune connection pools.
- Implement circuit breakers.
- Use read replicas.
- Enable automatic failover.
- Monitor connection health.

22. What testing should occur before maintenance?

Answer:

- Database connectivity tests.
- Failover testing.
- SSL validation.
- Performance testing.
- Connection pool testing.
- Application smoke tests.
- Rollback testing.

23. What should be documented in a Database RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- Affected applications.
- Database metrics.

- Configuration changes.
- Resolution steps.
- Rollback actions.
- Lessons learned.
- Preventive actions.

24. How would you improve database observability?

Answer:

- Monitor database metrics.
- Enable slow query logging.
- Track connection pool metrics.
- Create dashboards.
- Configure proactive alerts.
- Implement distributed tracing for database calls.

25. What preventive actions would reduce future incidents?

Answer:

- Monitor connection pool usage.
- Optimize queries regularly.
- Enable database and application health checks.
- Automate credential rotation with validation.
- Test failover periodically.
- Maintain SSL certificate lifecycle.
- Review Security Groups before deployment.
- Load test applications.
- Enable proactive alerts for latency and connection errors.
- Document rollback procedures.

Q: What is your approach to troubleshooting database connectivity issues in production?

Answer:

"I first determine whether the issue is affecting a single application or multiple services. I verify that the database is available and check basic network connectivity, DNS resolution, Security Groups, and firewall rules. Next, I validate authentication, connection strings, secrets, and SSL configuration. I review application logs, database logs, and monitoring dashboards to analyze connection counts, query latency, and resource utilization. If the issue started after a deployment or maintenance activity, I compare recent configuration changes and roll back if necessary. Once connectivity is restored, I perform a root cause analysis and implement preventive measures such as optimized connection pooling, proactive monitoring, failover testing, secure credential management, and regular performance tuning."

Test database port

nc -zv <db-endpoint> 5432 # PostgreSQL

nc -zv <db-endpoint> 3306 # MySQL

DNS lookup

nslookup <db-endpoint>

dig <db-endpoint>

PostgreSQL connection test

psql -h <db-endpoint> -U <user> -d <database>

MySQL connection test

mysql -h <db-endpoint> -u <user> -p

Check active PostgreSQL connections

SELECT * FROM pg_stat_activity;

Check MySQL process list

SHOW PROCESSLIST;

Kubernetes pod logs

kubectl logs <pod-name>

Resource usage

kubectl top pod

kubectl top node

AWS RDS status

aws rds describe-db-instances

1. HPA stops scaling during peak traffic. How would you investigate?

Answer:

- Check HPA status (kubectl describe hpa).
- Verify Metrics Server is healthy.

- Check CPU/Memory metrics.
- Review min/max replica limits.
- Verify target utilization thresholds.

2. Pods scale out but application remains slow. What would you check?

Answer:

- Verify Load Balancer distributes traffic.
- Check database bottlenecks.
- Review application logs.
- Validate readiness probes.
- Check external dependencies (cache, APIs).

3. Cluster Autoscaler doesn't add nodes. How would you troubleshoot?

Answer:

- Check Cluster Autoscaler logs.
- Verify pending pods.
- Review Auto Scaling Group/Node Group limits.
- Check cloud quotas.
- Validate node selectors and taints.

4. Scaling causes database connection failures. How would you investigate?

Answer:

- Check connection pool limits.
- Monitor active DB connections.
- Review database capacity.
- Validate application retry logic.
- Consider using a connection pooler (e.g., PgBouncer).

5. New replicas fail readiness checks. What would you analyze?

Answer:

- Review application startup logs.
- Check readiness probe configuration.

- Verify dependencies (DB, cache).
- Confirm resource availability.
- Check startup time.

6. CPU metrics are unavailable for HPA. How would you troubleshoot?

Answer:

- Verify Metrics Server is running.
- Check `kubectl top`.
- Review Metrics Server logs.
- Confirm RBAC permissions.
- Validate API aggregation layer.

7. Pods scale continuously without traffic changes. What would you investigate?

Answer:

- Review HPA thresholds.
- Check metric fluctuations.
- Verify application CPU spikes.
- Look for memory leaks.
- Review HPA stabilization settings.

8. Scaling increases cloud costs unexpectedly. How would you analyze them?

Answer:

- Review HPA and Cluster Autoscaler activity.
- Check over-provisioning.
- Analyze workload patterns.
- Right-size requests/limits.
- Optimize autoscaling policies.

9. Metrics Server stops responding. What would you check?

Answer:

- Verify Metrics Server pod status.
- Review logs.

- Check API Server connectivity.
- Validate certificates.
- Restart Metrics Server if required.

10. Scaling succeeds but traffic isn't distributed. How would you troubleshoot?

Answer:

- Check Service endpoints.
- Verify Load Balancer.
- Review Ingress configuration.
- Confirm readiness probes.
- Validate session affinity settings.

11. Node limits prevent scaling. How would you investigate?

Answer:

- Check cloud quotas.
- Review Node Group max size.
- Verify Auto Scaling limits.
- Check resource requests.
- Analyze pending pods.

12. Monitoring shows healthy clusters but users face delays. What would you analyze?

Answer:

- Review application latency.
- Check Load Balancer metrics.
- Analyze database performance.
- Verify request distribution.
- Review traces and logs.

13. Deployments fail during scale-out events. How would you troubleshoot?

Answer:

- Review deployment events.

- Check scheduling failures.
- Verify resource quotas.
- Analyze image pull issues.
- Review rollout strategy.

14. Resource quotas block scaling. What would you investigate?

Answer:

- Review ResourceQuota.
- Check namespace usage.
- Verify LimitRanges.
- Increase quotas if justified.
- Optimize resource requests.

15. Scaling creates intermittent outages. How would you analyze them?

Answer:

- Review rollout strategy.
- Check Pod Disruption Budgets.
- Analyze readiness/liveness probes.
- Verify traffic routing.
- Review HPA timing.

16. A scaling issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- HPA events
- Cluster Autoscaler logs
- Metrics Server logs
- Pod events
- Deployment history
- Node status
- Prometheus/Grafana metrics
- Application logs

- Timeline
- Recent changes

17. Which Kubernetes metrics are most useful?

Answer:

- CPU Utilization
- Memory Utilization
- HPA Desired vs Current Replicas
- Pod Pending Count
- Pod Restart Count
- Node CPU/Memory
- Request Latency
- Error Rate
- Cluster Autoscaler Metrics

18. What commands would you execute first?

Answer:

```
kubectl get hpa
kubectl describe hpa <hpa-name>
kubectl top pods
kubectl top nodes
kubectl get pods
kubectl describe pod <pod-name>
kubectl get events --sort-by=.metadata.creationTimestamp
kubectl get deploy
kubectl logs -n kube-system deployment/metrics-server
```

19. How would you validate scaling before production?

Answer:

- Perform load testing.
- Validate HPA behavior.
- Test Cluster Autoscaler.
- Verify readiness/liveness probes.

- Test database capacity.
- Conduct end-to-end smoke tests.

20. What controls reduce scaling risks?

Answer:

- HPA with appropriate thresholds.
- Cluster Autoscaler.
- Pod Disruption Budgets.
- Resource requests and limits.
- ResourceQuotas.
- Infrastructure as Code.
- Monitoring and alerts.

21. How would you improve scalability?

Answer:

- Right-size resources.
- Optimize HPA thresholds.
- Enable Cluster Autoscaler.
- Cache frequently accessed data.
- Optimize database connections.
- Use stateless application design.

22. What testing should occur before peak traffic?

Answer:

- Load testing.
- Stress testing.
- Spike testing.
- Failover testing.
- Autoscaling validation.
- Database performance testing.
- Disaster recovery drills.

23. What should be documented in a Scaling RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- Scaling behavior.
- HPA and Autoscaler events.
- Impact analysis.
- Resolution.
- Rollback actions.
- Lessons learned.
- Preventive actions.

24. How would you improve capacity planning?

Answer:

- Analyze historical trends.
- Forecast peak usage.
- Monitor resource utilization.
- Review scaling history.
- Regularly tune requests and limits.
- Plan cloud quotas in advance.

25. What preventive actions would reduce future incidents?

Answer:

- Tune HPA thresholds regularly.
- Right-size CPU and memory requests/limits.
- Monitor Metrics Server and Cluster Autoscaler.
- Set appropriate ResourceQuotas.
- Perform periodic load and stress tests.
- Optimize database connection pools.

- Configure proactive alerts.
- Review scaling policies after major releases.
- Conduct regular capacity planning.
- Maintain documented rollback procedures.

Q: What is your approach to troubleshooting Kubernetes scaling issues in production?

Answer:

"I first identify whether the issue is with Horizontal Pod Autoscaler (HPA), Cluster Autoscaler, application readiness, or infrastructure capacity. I review HPA status, Metrics Server health, and current CPU/memory metrics using `kubectl describe hpa` and `kubectl top`. If pods are pending, I check scheduler events, node capacity, resource quotas, taints, and Cluster Autoscaler logs. If pods are running but performance is poor, I investigate application bottlenecks such as database connections, caching, load balancer distribution, and readiness probes. I correlate recent deployments, configuration changes, and monitoring data to identify the root cause. Once the service is restored, I document the RCA, fine-tune autoscaling thresholds, optimize resource requests and limits, validate scaling through load testing, and improve capacity planning and monitoring to prevent recurrence."

HPA status

```
kubectl get hpa
```

```
kubectl describe hpa <hpa-name>
```

Resource usage

```
kubectl top pods
```

```
kubectl top nodes
```

Pod and deployment status

```
kubectl get pods
```

```
kubectl get deploy
```

```
kubectl describe pod <pod-name>
```

Events

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Metrics Server

```
kubectl get pods -n kube-system
```

```
kubectl logs -n kube-system deployment/metrics-server
```

Cluster Autoscaler (if deployed)

```
kubectl logs -n kube-system deployment/cluster-autoscaler
```

```
# Node information
kubectl get nodes -o wide
kubectl describe node <node-name>
```

1. An Ansible playbook fails halfway. How would you investigate?

Answer:

- Review the failed task output.
- Check the playbook logs (-vvv).
- Identify the failed host.
- Verify SSH connectivity and permissions.
- Re-run from the failed task using --start-at-task.

2. Configuration changes affect only some servers. What would you check?

Answer:

- Verify inventory groups.
- Check host variables.
- Review conditional (when) statements.
- Validate failed/skipped tasks.
- Compare configurations across servers.

3. Inventory files contain incorrect hosts. How would you troubleshoot?

Answer:

- Validate inventory syntax.
- Run ansible-inventory --list.
- Check group memberships.
- Verify dynamic inventory source.
- Confirm DNS/IP mappings.

4. SSH authentication fails during execution. How would you investigate?

Answer:

- Verify SSH keys.
- Check user permissions.
- Confirm host reachability.

- Review SSH configuration.
- Test manually using ssh.

5. A role update causes production issues. How would you troubleshoot?

Answer:

- Compare role versions.
- Review recent changes.
- Test the role in staging.
- Roll back to the previous version.
- Validate affected tasks.

6. Variables are applied incorrectly. What would you analyze?

Answer:

- Check variable precedence.
- Review group_vars and host_vars.
- Validate extra-vars.
- Inspect defaults and role variables.
- Use ansible -m debug to print variable values.

7. Idempotency fails after repeated executions. How would you investigate?

Answer:

- Identify tasks reporting unnecessary changes.
- Review module usage.
- Check file templates.
- Ensure commands use creates or removes.
- Prefer idempotent modules over shell commands.

8. Playbooks succeed but applications fail. What would you check?

Answer:

- Verify application logs.
- Check service status.
- Validate configuration files.

- Confirm dependencies are running.
- Test application health endpoint.

9. Multiple environments receive incorrect configurations. How would you troubleshoot?

Answer:

- Verify inventory selection.
- Check environment-specific variables.
- Review CI/CD pipeline parameters.
- Validate playbook targeting.
- Confirm branch and configuration version.

10. Ansible Vault secrets fail to decrypt. What would you investigate?

Answer:

- Verify Vault password.
- Check Vault ID.
- Validate encrypted files.
- Confirm CI/CD secret configuration.
- Test using ansible-vault view.

11. Parallel execution impacts production. How would you analyze it?

Answer:

- Review forks setting.
- Check serial configuration.
- Analyze server load.
- Execute rolling updates.
- Limit concurrency for critical services.

12. Package updates fail on selected servers. What would you check?

Answer:

- Verify repository availability.
- Check package manager logs.

- Validate OS compatibility.
- Review network connectivity.
- Confirm sufficient disk space.

13. Rollback automation doesn't work. How would you troubleshoot?

Answer:

- Review rollback playbook.
- Verify backups exist.
- Check failed rollback tasks.
- Test rollback in staging.
- Validate application after rollback.

14. Monitoring shows healthy servers but applications fail. What would you investigate?

Answer:

- Review application logs.
- Check configuration files.
- Validate service health.
- Verify database connectivity.
- Confirm load balancer routing.

15. Configuration changes increase downtime. How would you analyze them?

Answer:

- Review deployment strategy.
- Check service restart behavior.
- Analyze rolling update configuration.
- Validate dependency sequencing.
- Compare before/after configurations.

16. An Ansible issue causes a Sev-1 outage. What evidence would you collect?

Answer:

- Ansible execution logs

- Playbook output
- Inventory files
- Variable files
- CI/CD logs
- Application logs
- Configuration diffs
- Timeline
- Git commits
- Rollback history

17. Which logs help during Ansible incidents?

Answer:

- Ansible execution logs
- CI/CD pipeline logs
- SSH logs
- System logs (journalctl)
- Application logs
- Package manager logs
- Audit logs

18. What commands would you execute first?

Answer:

```
ansible all -m ping
ansible-inventory --list
ansible-playbook playbook.yml --check
ansible-playbook playbook.yml --diff
ansible-playbook playbook.yml -vvv
ansible-vault view secrets.yml
ansible all -m setup
```

19. How would you validate playbooks before production?

Answer:

- Run syntax check (--syntax-check).

- Execute in check mode (--check).
- Review changes with --diff.
- Test in staging.
- Peer review playbooks.
- Validate rollback.

20. What controls reduce automation risks?

Answer:

- Version control (Git).
- Code reviews.
- Ansible Vault.
- CI/CD validation.
- Role-based access control (RBAC).
- Environment separation.
- Rolling deployments.

21. How would you improve automation reliability?

Answer:

- Write idempotent playbooks.
- Modularize using roles.
- Implement error handling (block, rescue, always).
- Validate inputs.
- Automate testing.
- Monitor deployments.

22. What testing should occur before deployments?

Answer:

- Syntax validation.
- Dry-run (--check).
- Staging deployment.
- Smoke testing.

- Regression testing.
- Rollback testing.
- Performance testing.

23. What should be documented in an Automation RCA?

Answer:

- Incident summary.
- Timeline.
- Root cause.
- Affected systems.
- Playbook version.
- Configuration changes.
- Resolution.
- Rollback actions.
- Lessons learned.
- Preventive actions.

24. How would you improve configuration visibility?

Answer:

- Version all playbooks in Git.
- Use Infrastructure as Code.
- Centralize inventories.
- Track configuration drift.
- Maintain change history.
- Build automation dashboards.

25. What preventive actions would reduce future incidents?

Answer:

- Validate playbooks in staging.
- Enforce code reviews.
- Use Ansible Vault for secrets.

- Implement rolling deployments (serial).
- Ensure playbooks are idempotent.
- Maintain inventory accuracy.
- Automate testing in CI/CD.
- Enable detailed logging.
- Maintain rollback playbooks.
- Conduct regular automation reviews.

Q: What is your approach to troubleshooting Ansible issues in production?

Answer:

"I first identify whether the failure is related to inventory, connectivity, variables, playbook logic, or the target systems. I review the Ansible execution logs with increased verbosity (-vvv) to identify the exact task and host where the failure occurred. Then I verify SSH connectivity, inventory configuration, variable precedence, and module execution. If the playbook completed but the application is still affected, I validate the application configuration, service status, and logs. For production incidents, I stop further automation, roll back using the last known good configuration if necessary, restore service, and then perform a root cause analysis. To prevent recurrence, I enforce code reviews, idempotent playbooks, staged deployments, automated testing, and robust rollback procedures."

Test connectivity

ansible all -m ping

Validate inventory

ansible-inventory --list

Syntax check

ansible-playbook playbook.yml --syntax-check

Dry run

ansible-playbook playbook.yml --check

Show configuration differences

ansible-playbook playbook.yml --diff

Verbose execution

ansible-playbook playbook.yml -vvv

Start from failed task

ansible-playbook playbook.yml --start-at-task="Task Name"

```
# View encrypted file
ansible-vault view secrets.yml
```

```
# Gather facts
ansible all -m setup
```

1. DNS resolve ho raha hai lekin traffic ALB tak nahi pahunch raha —sabse pehle kya check karoge?

Answer:

- Verify ALB status.
- Test ALB DNS directly.
- Check Security Groups.
- Verify Listener configuration.
- Review Route53 alias record.
- Check VPC Flow Logs.

2. Listener rules aur Target Group mapping kaise verify karoge?

Answer:

- Check Listener configuration.
- Verify default action.
- Validate Host/Header and Path rules.
- Ensure correct Target Group association.
- Review Target Group health.

3. Target Group healthy hai lekin phir bhi requests fail ho rahi hain —kya dekhoge?

Answer:

- Listener rules.
- Application logs.
- Security Groups.
- WAF rules.
- Backend response codes.
- SSL configuration.

4. Security Groups ya NACLs traffic silently drop kar rahe hon toh kaise isolate karoge?

Answer:

- Verify inbound/outbound rules.
- Check ephemeral ports in NACL.
- Review VPC Flow Logs.
- Test connectivity using nc or telnet.
- Compare working and failing subnets.

5. HTTPS listener me certificate mismatch ka issue kaise detect karoge?

Answer:

- Verify ACM certificate.
- Check CN/SAN values.
- Validate Listener certificate attachment.
- Test using openssl s_client.
- Check browser SSL errors.

6. Cross-zone load balancing disable hone ka effect traffic routing par kaise trace karoge?

Answer:

- Check Cross-Zone setting.
- Compare target distribution across AZs.
- Review CloudWatch RequestCount per AZ.
- Identify if one AZ is overloaded.

7. Host-header ya path-based routing rules galat configure hon toh kya symptoms dikhenge?

Answer:

- 404/503 errors.
- Wrong application served.
- Default Target Group receives traffic.
- Healthy targets but incorrect routing.

8. Internal traffic ke liye Route Tables ya VPC Endpoints ka role kaise verify karoge?

Answer:

- Verify Route Tables.
- Check subnet association.
- Validate VPC Endpoint configuration.
- Review Security Groups.
- Test internal connectivity.

9. DNS TTL caching ki wajah se stale IP resolve ho raha ho toh kaise pehchanoge?

Answer:

- Check DNS TTL.
- Compare dig results.
- Flush DNS cache.
- Test from multiple clients.
- Compare Route53 records.

10. Route53 health checks aur ALB health checks mismatch ho toh kya troubleshoot karoge?

Answer:

- Compare health check paths.
- Verify thresholds.
- Check intervals.
- Validate backend health endpoint.
- Review Route53 health status.

11. Multi-region setup me DNS failover kaam nahi kar raha—kya check karoge?

Answer:

- Route53 failover policy.
- Health checks.
- Secondary endpoint.
- DNS propagation.

- CloudWatch alarms.

12. Client-side DNS caching traffic issue create kar raha ho toh kaise confirm karoge?

Answer:

- Flush DNS cache.
- Test from another client.
- Compare resolved IP.
- Check TTL.
- Use dig and nslookup.

13. ALB ka DNS name resolve ho raha hai lekin connection timeout aa raha hai— kya karoge?

Answer:

- Check ALB status.
- Verify Security Groups.
- Review Listener ports.
- Check Target Group.
- Review NACLs and VPC Flow Logs.

14. VPC Peering ya Transit Gateway routing traffic block kar raha ho toh kaise debug karoge?

Answer:

- Verify Route Tables.
- Check Peering/TGW attachments.
- Review Security Groups.
- Validate NACLs.
- Analyze VPC Flow Logs.

15. Application Load Balancer aur Network Load Balancer ke DNS behavior me farak kaise samjhoge?

Answer:

ALB

- Layer 7 (HTTP/HTTPS)
- Host/Path routing
- SSL termination

NLB

- Layer 4 (TCP/UDP)
- Static IP support
- Lower latency
- Preserves client IP

16. Kis tarah confirm karoge ki issue DNS layer par hai ya application layer par?

Answer:

- Resolve DNS using dig.
- Access ALB directly.
- Test backend Target Group.
- Compare HTTP response.
- Review application logs.

17. CloudWatch mein kaunse metrics DNS-vs-application failure differentiate karne mein help karte hain?

Answer:

- RequestCount
- HealthyHostCount
- TargetResponseTime
- HTTPCode_ELB_5XX
- HTTPCode_Target_5XX
- UnHealthyHostCount
- Route53 Health Check Status

18. VPC Flow Logs kaise use karoge traffic drop location identify karne ke liye?

Answer:

- Check ACCEPT/REJECT entries.

- Verify source/destination IP.
- Analyze destination port.
- Compare successful vs failed traffic.
- Identify Security Group/NACL block.

19. Client se ALB tak packet loss check karne ke liye kaunse tools use karoge?

Answer:

- ping (if applicable)
- traceroute
- mtr
- curl
- tcpdump
- Wireshark

20. DNS resolution successful hai lekin SSL handshake fail ho raha hai — kya investigate karoge?

Answer:

- Certificate validity.
- Certificate chain.
- ACM configuration.
- TLS versions.
- Cipher suites.
- Listener configuration.

21. Multi-AZ deployment me ek AZ ka traffic fail ho raha ho toh kya check karoge?

Answer:

- Target health by AZ.
- Cross-zone load balancing.
- Subnet mapping.
- Route Tables.
- Auto Scaling.

- Application health.

22. Load Balancer ke saamne WAF configured ho aur traffic block ho raha ho toh kaise verify karoge?

Answer:

- Review WAF logs.
- Check blocked requests.
- Verify Web ACL rules.
- Test from different IPs.
- Compare allowed vs blocked traffic.

23. Is scenario ka RCA likhte waqt kya evidence collect karoge?

Answer:

- Route53 records
- ALB Listener configuration
- Target Group health
- Security Groups
- NACL configuration
- VPC Flow Logs
- WAF logs
- CloudWatch metrics
- Application logs
- Timeline
- Deployment history

24. Future me is tarah ke issues avoid karne ke liye kaunse preventive controls lagaoge?

Answer:

- Infrastructure as Code (Terraform).
- Automated health checks.
- DNS monitoring.
- ALB monitoring.

- WAF monitoring.
- Change approvals.
- Staging validation.
- Canary deployments.
- Route53 health checks.
- Automated rollback.

25. DNS aur traffic routing ki observability improve karne ke liye kya best practices follow karoge?

Answer:

- Enable Route53 Health Checks.
- Monitor ALB CloudWatch metrics.
- Enable Access Logs.
- Use VPC Flow Logs.
- Centralize logs.
- Synthetic monitoring.
- Distributed tracing.
- Alert on DNS failures.
- Dashboard for traffic flow.
- Monitor latency by AZ.

Q: Production me DNS resolve ho raha hai lekin application accessible nahi hai. Aap kaise troubleshoot karoge?

Answer:

“Main issue ko layer-by-layer isolate karta hoon. Sabse pehle dig ya nslookup se confirm karta hoon ki DNS correct ALB endpoint resolve kar raha hai. Phir ALB ki health, listeners, listener rules aur target groups verify karta hoon. Agar target healthy hain to Security Groups, NACLs, Route Tables aur VPC Flow Logs check karta hoon ki traffic kahin drop to nahi ho raha. HTTPS case me ACM certificate, TLS handshake aur WAF logs validate karta hoon. Agar ALB tak traffic aa raha hai lekin backend fail ho raha hai to application logs, health endpoints aur deployment changes analyze karta hoon. Service restore hone ke baad RCA prepare karta hoon aur preventive measures jaise Route53

health checks, CloudWatch alerts, synthetic monitoring, Infrastructure as Code, aur canary deployments implement karta hoon."

DNS resolution

dig app.example.com

nslookup app.example.com

Test HTTPS endpoint

curl -lv <https://app.example.com>

Test TLS handshake

openssl s_client -connect app.example.com:443

Check ALB Target Health

aws elbv2 describe-target-health \

--target-group-arn <target-group-arn>

Describe ALB listeners

aws elbv2 describe-listeners \

--load-balancer-arn <alb-arn>

Check Route53 record

aws route53 list-resource-record-sets \

--hosted-zone-id <hosted-zone-id>

Network connectivity

tracert app.example.com

mtr app.example.com

Kubernetes ingress (if ALB Ingress Controller is used)

kubectl get ingress

kubectl describe ingress <ingress-name>